

Algorithms & Data Structures

Exercise Sheets

Weeks 7-8: Heaps and Binary Search Trees

Submit your solutions via feedback fruits in Brightspace, Week 7-8. Your handin should consist of a **zip file containing a pdf and any code** you were required to produce, along with the main files and dependencies to allow the instructor to compile and test your code. In the pdf please include **screenshots of the most important parts of the code**.

Exercises

- (1) (Week 7) Using pen and paper, illustrate how a *min-heap* is built from the array

[59, 44, 79, 17, 54, 32, 31, 12, 7, 4, 1]

using *heapify*. You must illustrate all steps of the heapify process. Then illustrate first what happens if *remove* is called on the resulting heap and then what happens if *insert(2)* is called on the heap (before the remove).

- (2) (Week 7) Based on min_heap.h, implement the min heap functionality based on the algorithm described in the slides, *using the binary tree node structure*:

```
template <typename T>
struct Node {
    T data;
    Node* parent;
    Node* left;
    Node* right;

    // Constructor
    Node(T value) {
        data = value;
        left = right = nullptr;
    }
};
```

Also provide a test program to test your implementation.

- (3) (Week 7) Based on the Priority Queue ADT in priority_queue.h, create an implementation, MinHeapPriorityQueue(), that uses your MinHeap implementation. Also, provide testing code.
- (4) (Week 8) Using pen and paper, illustrate how an AVL tree is built with the following elements [10, 20, 15, 25, 30, 16, 18, 19] (i.e. 10 is inserted first, then 20 ...). You must illustrate all rotations etc. Then illustrate what happens when 30 is deleted and then 18
- (5) (Week 8) A node in a binary tree is an only-child if it has a parent node but no sibling node (Note: The root does **not** qualify as an only child).

The *loneliness-ratio* (*LR*) of a given binary tree *T* is defined as the following ratio:

$$LR(T) = \frac{\text{The number of nodes in } T \text{ that are only children}}{\text{The number of nodes in } T} \quad (1)$$

- (a) Argue that for any nonempty AVL tree T , we have that $LR(T) \leq 1/2$
 - (b) Is it true for any binary tree T , that if $LR(T) \leq 1/2$ then $height(T) = \log(n)$?
- (6) (Week 8) The `OrderedSet` (see: `ordered_set`) is a container that contains data in increasing order and that does not allow duplicates. Write an implementation for the `OrderedSet` class using the `BinarySearchTree` implementation we saw in class as the internal data structure. `BinarySearchTree` and test program to test the set can be found in the repo. You will have to implement all the functions marked as `TO BE IMPLEMENTED` in the `ordered_set.h` and `binary_search_tree.h` files, see details next.

Most of the functions in the `OrderedSet` class can be implemented by calling the corresponding functions in the `BinarySearchTree` class. A few of them may require some additional work.

- `insert` function returns an iterator to that either represents the newly inserted item or the existing duplicate item that is already stored in the container.
- `find` searches for a value and returns an iterator representing the location of the item (or the endmarker if the search fails).
- `erase` removes the object at the position given by the iterator and returns an iterator representing the element that followed `itr` immediately prior to the call to `erase`. Finally it invalidates `itr`.
- `++` operator must be implemented for the iterator class in `BinarySearchTree` to find the next element (in increasing order) This is needed for the implementation of `erase`.
- `begin` function must return an iterator to the smallest elements.
- `end` function must return an iterator to one past the largest element (C++ convention).
- `OrderedSet(const OrderedSet& s)` is the copy constructor to make a deep copy of an existing set.

Use the debugger to resolve any issues you may encounter.

- (7) (Week 8) Design and implement a linear-time algorithm that verifies that the height information in an AVL tree is correct i.e. the balance property is in order for all nodes.